DD MM AA

OO Modeling & Conceptual Modeling Practices

Conceptual modeling captures ideas and concepts to analyze, specify, design, and efficiently communicate real-world software structures and behaviors.

Key OO Design Principles

- Layered Architecture: Software should be organized into loosely coupled layers (e.g., User Interfaces, Application Logic, Persistence, Communications, OS), where higher layers consume lower-layer abstractions.
- Aggregation Over Inheritance: Prefers aggregation over inheritance for code reuse because composition/aggregation relationships can change dynamically at runtime, yielding greater flexibility.
- Interface-Driven Design: Program to interfaces or abstractions to reduce coupling, manage dependencies, and eliminate accidental complexity.
- Naming conventions. Class names should be singular nouns in Pascal case/Upper Camel Case (eg. Student), variables in lower camel case, and methods/operations in lower camel case verb phrases.
- Unit testing: Executable unit tests verify correctness, reveal design flaws, enable regression testing, and document code usage.

Introduction to the Unified Modeling Language (UML)

Nature of UML: UML is an expressive, semi formal, process-independent, and extensible visual modeling language designed to support incremental software development.

Building Blocks & Organization

- Elements: Modeling elements (structural, behavioral, organizational, and annotational) act like "words".
- Diagrams: Visual representations combining elements act like "sentences" or "paragraphs".

Norma
Escaneado con CamScanner

Models & Views: Views group related diagrams to describe specific perspectives (eg., Use case, Logical, Implementation, Process, Deployment views).

Three Software Engineering Perspectives.

Analysis: Focusses on understanding the problem domain using real-world concepts.

Specification (High-Level Design): Maps out the logical software system, interfaces, and automation boundaries

Implementation: Describes actual software constructs, language types, libraries, and frame works.

Diagram classification

Structural Diagrams: Class, Object, Components and Deployment diagrams illustrate static system structures and components.

Behavioral Diagrams: Use case, sequence, comunication (collaboration), state chart, and Activity diagrams capture dynamic system behaviors, workflows, and interactions.

Use Case Diagrams

Actors: External entities (human users, hardware, or external software systems) that interact with the system to archieve specific goals.

Use Cases: Represent discrete user-visible goals and functionality independent of internal software features ("what" and "why" rather than "how").

Relationships.

<<include>> Indicates that a base use case explicitly requires and integrates the sub-goal of another use case.

<<extend>> Represents optional variations or conditional extensions to a base use case at defined extension points.

Escaneado con CamScanner

Generalization. Specifies a specialized use case (or actor) that is a subset/refinement of a broader one.

Objects, Classes, and class Diagrams

Objects are individual run time entities encapsulating state, behaviors and identity; classes are abstract constructs that group collections of objects sharing common properties

Class syntax: Standard 3-compartment boxes containing.

1. Class Name: Singular noun in UpperCamelCase
2. Attributes: Properties formatted as

   [visibility] name [multiplicity] [: type] [= initial-value].
3. Operations: Behaviors formatted as

   [visibility] name [(parameter-list)] [: return-type].
4. Optional compartments: Responsibilities, constraints or user defined notes.

Relationships in Class Diagrams

Generalization: An "is-a" relationship representating inheritance/subtyping (shown with a hollow arrowhead).

Association: Structural connection between classes ("has-a" or links), which can be binary or N-ary, and include navigation arrows and multiplicity constraints (e.g. 1, 0..*, 1..*).

Aggregation: A weak "part-of" association represented by an open diamond.

Composition: A strong "part-of" aggregation represented by a solid diamond, where the life cycle of the part is strictly bound to the aggregate whole.

Dependency: A "uses" relationship represented by a dashed arrow, indicating that changes in one class may affect another.

Escaneado con CamScanner

DD MM AA

UML Quick Reference Guide Summary

Model workflows using Initial Nodes, Action Nodes, Control / Object Flows, Decision / Merge Nodes, Fork / Join Nodes, Signal Actions, and Final Nodes.

Interaction Diagrams:

Communication Diagrams: Emphasize structural connections and message pathways between objects.

Sequence Diagrams: Focus on time-ordered exchange of messages along object lifelines (supports synchronous, asynchronous, creation, and reply messages)

Model state transitions, triggers, guard conditions, and internal actions (entry, exit, do activities) over an objects lifecycle.

Escaneado con CamScanner